A Survey of Fine-Tuning Frameworks and Reinforcement Learning Tooling for Speech-to-Text

Gautam Agarwal*, Shivangi Kumar*, Chen Wang†
*Department of Computer Science, Columbia University, New York, USA
Email: {first.last}@example.edu
†IBM Research Center, New York, USA
Email: second@example.com

***Abstract*—**
***Index Terms*—Speech-to-Text, Reinforcement Learning, RLHF, Continuous Learning, Fine-Tuning, NeMo, ESPNet**

I. Introduction

A. Motivation and Problem Context

Modern speech-to-text (STT) systems have achieved impressive performance on standard benchmarks, yet their accuracy degrades substantially when deployed in real-world environments characterized by domain shift, background noise, and diverse speaker accents. Production audio streams often differ markedly from training corpora along dimensions such as vocabulary, speaking style, channel characteristics, and acoustic conditions, leading to rising word error rates (WER) and unstable user experience over time. This mismatch is exacerbated in specialized settings—for example, clinical, legal, or call-center domains—where terminology, discourse structure, and recording setups are highly domain-specific.

The dominant operational paradigm today is to treat STT models as static assets: models are pre-trained once on large general-purpose corpora, optionally fine-tuned on a fixed domain dataset, and then frozen for deployment. When performance eventually drifts, organizations resort to periodic manual retraining cycles that require costly data curation, human annotation, and engineering effort to refresh checkpoints. Such workflows are reactive rather than proactive, incur substantial latency between error detection and remediation, and often ignore many potentially informative signals available in production (e.g., user corrections or downstream task failures).

Crucially, most deployed STT pipelines lack a closed-loop mechanism by which the system can autonomously detect its own systematic errors, turn these errors into training signals, and schedule fine-tuning in a way that balances accuracy gains against computational cost. While recent work on agentic and self-learning systems has demonstrated the feasibility of generate–critique–improve loops in other modalities, analogous architectures for STT remain underexplored, and existing tools offer limited support for truly self-improving speech recognition in production environments.

B. Scope and Survey Objectives

This survey focuses on the fine-tuning and adaptation capabilities of two representative STT toolchains, NVIDIA NeMo and ESPnet/ESPnet2, which provide configurable recipes, pre-trained acoustic and sequence-to-sequence models, and parameter-efficient fine-tuning mechanisms that are widely used in research and practice. We examine how each framework structures its training pipeline, exposes configuration hooks, and supports domain adaptation workflows relevant to long-lived STT deployments. In particular, we pay attention to configuration-driven design patterns that either facilitate or hinder continual and cost-aware adaptation in production settings.

Complementing this STT-centric view, we conduct a high-level scan of modern reinforcement learning and reinforcement learning from human or AI feedback (RL/RLHF) frameworks, including VERL, Hugging Face TRL, DeepSpeed-Chat (DeepSpeed-RLHF), OpenRLHF, TRLX, RL4LMs, and Ray RLlib, among others. These libraries provide algorithmic and systems infrastructure for on-policy and off-policy optimization (e.g., PPO- and GRPO-style methods, preference optimization) that has so far been applied primarily to large language models rather than speech recognition. We also briefly consider speech language model alignment stacks such as Align-SLM and SpeechLM, which implement preference-based training for speech-to-speech or speech-conditioned generation and offer additional design signals for feedback-driven optimization on speech.

Our goal is not to provide an exhaustive survey of RLHF or speech language modeling, but to assess to what extent these RL/RLHF and speech-LM alignment frameworks can be repurposed to drive self-adaptive STT. Throughout the paper, the emphasis is on identifying design gaps and integration challenges rather than proposing a single, fully specified new solution. In particular, we analyze how current STT toolchains and RL/RLHF tooling fall short of providing out-of-the-box support for closed-loop, feedback-driven speech systems and distill a set of design requirements that any future self-learning STT framework should satisfy.

C. Contributions and Paper Organization

This survey makes four primary contributions. First, we develop a taxonomy of STT fine-tuning frameworks along dimensions such as model architectures (CTC, transducer, encoder–decoder), adaptation strategies (full fine-tuning, parameter-efficient fine-tuning, domain and speaker

adaptation), data and configuration pipelines, and extensibility of the training loop. This taxonomy highlights common design patterns and exposes where existing frameworks natively support, or actively hinder, more dynamic learning schemes.

Second, we propose a conceptual mapping between STT toolchains and RL/RLHF components, viewing ASR frameworks as environments and policies whose behaviors could, in principle, be optimized by external controllers. We identify how elements such as data loaders, loss functions, beam search decoders, and evaluation metrics relate to RL notions of states, actions, rewards, and rollout buffers, and we discuss where current APIs provide or lack the hooks required to instantiate such a mapping. We draw on speech-LM alignment systems such as Align-SLM and SpeechLM only as illustrative examples of how preference-based objectives over speech signals might be integrated into such a mapping.

Third, we outline an experimental protocol for comparing NeMo and ESPnet on shared datasets and evaluation metrics. The protocol spans conventional supervised fine-tuning as well as continual-learning scenarios that simulate domain shift, leveraging a unified evaluation stack with metrics such as WER, character error rate (CER), diarization error rate (DER), and domain-specific error measures. These experiments, conducted on both public and application-specific corpora, provide empirical evidence about the strengths and limitations of each framework as a substrate for self-adaptive STT.

Finally, based on our taxonomy, mapping, and experiments, we perform a systematic gap analysis that surfaces missing capabilities at the intersection of STT fine-tuning and RL/RLHF tooling. We articulate open research questions and design opportunities, arguing that an effective self-adaptive STT framework must integrate replay-based continual learning, reward-driven objectives over rich speech and semantic metrics, and cost-aware scheduling into a unified, production-ready stack. We deliberately leave the concrete instantiation of such a framework to future work, which this survey is intended to motivate.

II. Background and Problem Setting

A. Modern STT Architectures, Their Strengths, and Fine-Tuning Motivation

Contemporary speech-to-text (STT) systems are built on a small number of canonical architectural families, each making different trade-offs between accuracy, latency, streaming capability, and adaptability. Understanding these trade-offs is essential not only for selecting a deployment architecture but also for understanding *why* and *how* fine-tuning must be applied differently depending on which family a model belongs to. We therefore review CTC, RNN-T, and attention-based encoder–decoder architectures in turn, characterise their strengths and limitations, and connect these properties to the fine-tuning strategies that follow in later sections.

Connectionist Temporal Classification (CTC): CTC models pair an acoustic encoder (typically a Conformer or Transformer stack) with a simple linear projection and a CTC loss that marginalises over all valid alignments between frame-level encoder outputs and the target token sequence, without requiring an explicit language model or autoregressive decoder.

Strengths. Because CTC decoding is non-autoregressive, inference is highly parallelisable and latency is low: the entire transcript can be emitted in a single forward pass through the encoder, making CTC models well-suited to real-time and streaming deployments where strict end-to-end latency budgets must be respected. CTC models are also relatively simple to train and fine-tune—the loss has a single hyperparameter (blank probability weight) and the decoder adds essentially no parameters, so adaptation on new domain data primarily means updating the encoder. This architectural simplicity makes CTC a common choice for on-device ASR, voice assistants, and latency-sensitive call-centre transcription.

Limitations. CTC relies on a conditional independence assumption that frames are emitted independently given the encoder states, preventing the model from capturing within-sequence language dependencies at the decoder level. In practice, this means CTC models underperform attention-based decoders on tasks requiring strong language priors, complex morphology, or rare domain vocabulary—exactly the conditions encountered in clinical, legal, and technical transcription settings. Without an external language model, CTC systems are also more sensitive to out-of-vocabulary terms, which directly motivates domain-adaptive fine-tuning to update the encoder's internal representations for new lexical distributions.

Recurrent Neural Network Transducer (RNN-T): The RNN-T (or more generally, the neural transducer) extends CTC with an autoregressive prediction network that conditions each output token on the previously emitted tokens, and a joint network that combines encoder and prediction states to score each emission.

Strengths. The autoregressive prediction network gives RNN-T models an internal language model that substantially improves accuracy on morphologically rich languages, domain-specific terminology, and conversational speech compared to pure CTC. Crucially, RNN-T supports chunk-wise streaming decoding: the encoder can process audio in fixed-size chunks while the prediction network maintains a running token history, enabling low-latency, word-by-word output that is essential for voice search, live captioning, and embedded devices. This combination of streaming capability and built-in language context makes RNN-T the dominant architecture in production ASR systems deployed by major technology companies.

Limitations. The joint network introduces additional memory and computation relative to CTC, and the RNN-T loss requires computing a probability lattice over the full encoder$\times$prediction grid, making training more expensive. The autoregressive decoder also couples prediction and acoustic states in a way that can make fine-tuning more sensitive to distribution shift: when domain vocabulary changes, both the encoder representations *and* the prediction network must be adapted, and mismatches between the two during partial fine-tuning can degrade performance more sharply than in

CTC. This coupling makes PEFT strategies for RNN-T—such as freezing the encoder and only adapting the prediction network—less reliable and requires careful layer-wise tuning protocols.

Attention-Based Encoder–Decoder (AED): Attention-based encoder–decoder (AED) models use a deep acoustic encoder (Conformer, Transformer, or Branchformer) together with a Transformer decoder that attends over all encoder outputs via cross-attention, generating transcript tokens autoregressively. Hybrid CTC/attention models blend a CTC loss on the encoder with the attention decoder objective, combining the fast alignment properties of CTC with the richer language model of an attention decoder.

Strengths. AED models consistently achieve the lowest word error rates on standard benchmarks due to their global attention mechanism, which allows the decoder to "look back" over the full encoded utterance before emitting each token. This global context is particularly valuable for long-form transcription, accented speech, and domain-specific text with complex multi-word expressions, where local acoustic context alone is insufficient. The hybrid CTC/attention formulation also improves training stability and allows the decoder to be guided by CTC alignments early in training, before attention mechanisms are fully learned.

Limitations. Because the attention decoder operates over the full encoded sequence, standard AED models cannot produce output until the entire utterance is encoded, making them unsuitable for true streaming applications with strict latency constraints without additional modifications such as monotonic attention or chunk-wise encoding. Autoregressive decoding is also inherently sequential, limiting inference throughput on long-form audio compared to CTC. Fine-tuning AED models is expensive relative to CTC: both the encoder and the multi-layer Transformer decoder contain large parameter counts, and naïve full fine-tuning risks catastrophic forgetting of the decoder's implicit language model when only limited in-domain data is available.

Conformer Encoder as the Shared Backbone: Across all three decoder families, the Conformer encoder has emerged as the dominant acoustic front-end. Its architecture interleaves multi-head self-attention layers with depthwise convolutional modules and macaron-style feed-forward blocks, enabling simultaneous modelling of long-range temporal dependencies (via attention) and local phonetic patterns (via convolution) that pure Transformer or pure CNN encoders capture only partially. NVIDIA's FastConformer variant introduces aggressive temporal subsampling through depthwise-separable convolutions, reducing the sequence length seen by the attention layers and substantially cutting memory and compute cost, while maintaining competitive WER on both short and long-form audio. In ESPnet2, the Branchformer encoder provides an alternative that uses parallel convolutional and attention branches within each block, offering architectural diversity with modest accuracy gains on certain benchmarks.

Architecture-to-Use-Case Mapping: Table I summarises the key trade-offs and recommended deployment contexts for each architectural family.

Why Architecture Determines Fine-Tuning Strategy: The architectural properties described above directly shape both the *necessity* and the *method* of fine-tuning in production settings.

CTC's encoder-only adaptation surface makes it the most tractable target for lightweight PEFT: because the decoder adds negligible parameters, techniques such as LoRA on encoder attention projections or adapter modules on feed-forward layers can capture domain-specific acoustic and lexical patterns with as few as 1–2 % of total parameters updated. However, CTC's lack of an internal language model means that even small vocabulary shifts—e.g., medical terminology in a clinical transcription deployment—can cause disproportionate WER degradation, making domain fine-tuning *necessary* rather than merely beneficial when CTC models are moved to specialised domains.

RNN-T models face a more complex fine-tuning landscape because the prediction network captures language statistics that may be well-calibrated on general corpora but poorly matched to narrow domains. Adapting only the encoder while freezing the prediction network can leave a mismatched prior, while full joint fine-tuning risks overfitting both components on small domain datasets. These trade-offs motivate staged fine-tuning recipes—first updating only the encoder, then jointly fine-tuning with a reduced learning rate—and architecture-aware PEFT designs that target each sub-network separately.

For AED models, the Transformer decoder's implicit language model is both an asset (global fluency, handling of rare morphological forms) and a liability when fine-tuning on limited domain data: the decoder can overfit to narrow phrase distributions while the encoder under-adapts to new acoustic conditions. Multi-stage fine-tuning that first freezes the decoder and adapts the encoder, then jointly fine-tunes with a low learning rate, is therefore the standard practice, with PEFT applied to decoder cross-attention and self-attention layers to reduce the risk of catastrophic forgetting.

Across all architectures, the scheduling problem—deciding when, on what data, and with what computational budget to trigger fine-tuning—is inseparable from the architectural properties outlined here. A CTC model may warrant frequent, lightweight encoder-only updates in response to vocabulary drift, while a large AED model may require less frequent but more comprehensive multi-stage adaptation cycles. This architecture-aware view of fine-tuning necessity and strategy provides the foundational motivation for the taxonomy, framework analyses, and RL/RLHF alignment discussions that follow in subsequent sections.

Standard Supervised Fine-Tuning Loop and Metrics: The standard supervised fine-tuning loop starts from a pre-trained acoustic or encoder–decoder model and adapts it on a target-domain dataset via mini-batch stochastic gradient descent on cross-entropy, CTC, or hybrid objectives. Typical domain adaptation recipes freeze part of the encoder, adjust learning-rate schedules (e.g., warm-up followed by inverse-square-root decay), and use reduced batch sizes and early stopping tuned to

TABLE I
STT ARCHITECTURE TRADE-OFFS AND PRIMARY USE CASES.

Architecture	Key Strengths	Key Limitations	Primary Use Cases
CTC	Fast, parallel decoding; low latency; simple fine-tuning	No internal LM; sensitive to OOV; weaker on complex domains	Real-time transcription; voice assistants; on-device ASR; call-centre STT
RNN-T	Streaming with built-in LM context; strong on morphologically rich/conversational speech	Expensive training; coupled encoder–decoder complicates domain PEFT	Production streaming ASR; voice search; live captioning; embedded devices
AED (Encoder–Decoder)	Highest accuracy; global context; strong on long-form and accented speech	Not natively streamable; slow autoregressive decoding; expensive full fine-tuning	Offline transcription; medical/legal dictation; subtitling; research benchmarks
Hybrid CTC/Attention	Best of CTC + AED; stable training; flexible trade-off via λ	Moderate latency; larger model than pure CTC	High-accuracy batch transcription; domain-adaptive research; ESPnet default recipe

downstream validation metrics. Evaluation relies on sequence-level metrics such as word error rate (WER) and character error rate (CER), often complemented by diarization error rate (DER), verb error rate, and domain-specific error measures in application-focused studies. Frameworks like ESPnet2 provide unified evaluation scripts that compute WER/CER on held-out sets and can log per-utterance errors, beam scores, and token-level information for subsequent analysis.

B. Reinforcement Learning and RLHF for Sequence Models

Reinforcement learning (RL) for sequence modeling treats the model as a conditional policy that generates token sequences in response to input contexts, with learning signals derived from a reward function defined over complete trajectories. Given an input x (e.g., an audio segment or prompt), a policy π_θ produces an output sequence $y = (y_1, \ldots, y_T)$, which is evaluated by a task-specific reward $R(x, y)$, and the objective is to maximise the expected return $\mathbb{E}_{y \sim \pi_\theta(\cdot|x)}[R(x, y)]$ via gradient-based updates. Policy gradient methods such as REINFORCE optimise this objective using Monte Carlo estimates of $\nabla_\theta \log \pi_\theta(y \mid x)$ weighted by rewards or advantages, optionally with variance-reduction baselines.

Reinforcement learning from human feedback (RLHF) and related paradigms adapt this RL view to align large sequence models with human or AI preferences. A typical RLHF pipeline begins with supervised fine-tuning (SFT) of a base model on instruction–response pairs to establish a reasonable initial policy. Next, a reward model is trained on pairwise preference data, where human or AI annotators indicate which of two responses is preferred, yielding a learned scalar reward function $r_\phi(x, y)$ that approximates latent quality judgments. Finally, on-policy RL algorithms such as proximal policy optimization (PPO) or group relative policy optimization (GRPO) iteratively sample rollouts from the current policy, score them with the reward model, and update the policy to increase expected reward while constraining divergence from a reference model.

Beyond PPO-style methods, recent work has explored direct preference optimization (DPO) and related objectives that optimise implicit likelihood ratios between preferred and dispreferred responses without an explicit policy-gradient loop. Reinforcement learning from AI feedback (RLAIF) further replaces human raters with strong teacher models that generate preference labels, enabling scalable alignment at the cost of potential bias transfer. Modern RLHF and RLAIF frameworks (e.g., VERL, OpenRLHF, DeepSpeed-Chat, TRL, TRLX, RL4LMs) provide implementations of these algorithms, together with infrastructure for distributed rollout generation, reward modeling, and multi-turn agentic workflows, primarily for text-based large language models.

C. Continuous and Lifelong Learning in STT

Continuous and lifelong learning aim to enable models to absorb new information over time while retaining performance on previously learned tasks or domains. In the STT setting, this corresponds to scenarios where deployed systems encounter evolving input distributions due to new accents, acoustic conditions, domains, or recording hardware, and must adapt without catastrophic forgetting of earlier conditions. Catastrophic forgetting arises when fine-tuning on new data dramatically degrades accuracy on earlier domains, while backward transfer measures whether learning on later tasks improves or harms performance on prior ones.

Replay buffers are a standard tool for mitigating forgetting by maintaining a curated subset of past examples that are interleaved with new data during retraining, approximating a mixture of historical and current distributions. In speech applications, such buffers can store audio–transcript pairs drawn from prior domains, synthetic "oracle" transcripts generated by large language models, or automatically corrected outputs from online post-editing modules. Lifelong-learning experiments for STT therefore often compare naive sequential fine-tuning against replay-enhanced strategies under controlled task sequences and drift protocols, tracking both in-domain and cross-domain WER trajectories as new data are introduced.

A central challenge in continuous STT learning is the scheduling problem: deciding when and how aggressively to fine-tune under compute, latency, and budget constraints. Frequent updates can reduce lag between distribution shift and adaptation but may be prohibitively expensive in GPU hours or cloud cost, whereas infrequent retraining risks prolonged periods of degraded user experience. Recent work has proposed adaptive schedulers that monitor validation metrics, cost efficiency, and signs of diminishing returns to trigger fine-tuning only when additional gains justify the computational expense, but such mechanisms are not yet integrated into mainstream STT toolkits. Bridging the gap between these continual-learning desiderata and the capabilities of existing ASR and RL/RLHF frameworks is therefore a key problem this survey seeks to characterise.

III. Survey Methodology and Evaluation Protocol

IV. Taxonomy of STT Fine-Tuning Frameworks

A. Taxonomy Dimensions

To reason systematically about the suitability of existing speech toolkits as substrates for self-adaptive STT, we first introduce a taxonomy along three primary dimensions: model support, adaptation strategies, and data/recipe ecosystems.

Model support. Frameworks differ in the families of acoustic and sequence models they support, as well as in their coverage of streaming versus offline decoding. NVIDIA NeMo provides Conformer and FastConformer encoders with both CTC and RNN-T decoders, supporting non-autoregressive transcription as well as streaming-capable transducer models configured via YAML files and Python scripts. ESPnet2 exposes a broader catalog of encoder–decoder ASR architectures, including Conformer, Transformer, Branchformer, transducer, and streaming Conformer variants, instantiated through task-specific recipes with configurable CTC/attention hybrids and language model integration. In this taxonomy we focus on NeMo and ESPnet2 as representative STT frameworks; speech language model alignment stacks such as Align-SLM and SpeechLM are discussed later as related speech-to-speech alignment toolkits rather than core STT systems.

Adaptation strategies. Both NeMo and ESPnet2 support supervised domain adaptation, but they differ in how fine-grained their control over parameter updates and schedules is. NeMo offers full fine-tuning as well as parameter-efficient fine-tuning (PEFT) mechanisms such as low-rank adaptation layers, adapter modules, and IA3-style rescaling, implemented by wrapping selected linear layers while leaving the base model frozen. ESPnet2 exposes adaptation through configuration-level controls that determine how pretrained checkpoints are loaded and which subsets of the model remain frozen during fine-tuning, together with choices of SpecAugment strength, front-end architectures for robustness, and warm-up learning-rate schedulers tuned for adaptation on smaller, domain-specific datasets. In both frameworks, multi-stage training recipes and layer-wise freezing patterns enable practitioners to trade off adaptation capacity, stability, and computational cost, although PEFT recipes for ASR remain less mature and standardised than for large language models.

Data and recipe ecosystems. Both NeMo and ESPnet2 provide rich recipe ecosystems that encode best practices for corpus preparation, model configuration, and evaluation. NeMo uses manifest-based data loaders (JSON with audio paths and transcripts), tarred dataset support, bucketing by utterance length, and centralized configuration files for specifying model, optimizer, and scheduler settings within a PyTorch Lightning training loop. ESPnet2 organises recipes by corpus (e.g., LibriSpeech, AISHELL), with shell scripts orchestrating multi-stage pipelines for data preparation, BPE training, ASR and LM training, checkpoint averaging, and decoding, backed by YAML configs that map directly into front-end, encoder, decoder, CTC, and optimizer constructors. These ecosystems effectively constitute opinionated MLOps templates for ASR research and deployment, but they were designed primarily for

supervised training and periodic offline adaptation rather than for tightly integrated, closed-loop self-learning.

B. RL Alignment and Control Hooks as a Taxonomy Dimension

For self-adaptive STT, a critical additional axis is the extent to which a framework exposes alignment and control hooks suitable for reinforcement learning or RLHF-style training. We characterise this along two sub-dimensions: reward injection points and training-loop modularity.

Reward injection and loss abstraction. In NeMo, ASR models implement loss computation inside task-specific modules that by default combine CTC or RNN-T criteria with optional auxiliary losses; replacing these with reward-based objectives requires overriding internal loss functions or extending model classes but is conceptually straightforward. ESPnet2 provides a more explicit scaffold by introducing reinforcement-learning-aware ASR variants and associated trainers, in which a policy-gradient loss term can be computed alongside the standard CTC/attention loss and blended via a scalar coefficient that controls the trade-off, with rewards derived from per-utterance metrics such as WER. In both frameworks, reward functions are currently ad hoc Python code rather than part of a unified reward API, and standard ASR recipes treat WER and CER as diagnostic outputs rather than as optimisable objectives.

Training-loop modularity, logging, and replay. From a control perspective, it is important that the training loop can be modularised into rollout, scoring, and update phases, and that it exposes sufficient logging for constructing replay buffers and offline analyses. NeMo relies on PyTorch Lightning trainers that encapsulate the core optimisation loop; while users can inject callbacks and custom hooks, on-policy RL or off-policy replay typically require more intrusive modifications to separate inference-only rollouts from gradient steps. ESPnet2's trainer is implemented at the framework level with clear separation between the core training routine and the inference stages, and custom inference utilities already log per-utterance hypotheses, scores, and WER to structured files, enabling reward computation and offline policy-gradient experiments without changing the trainer. More advanced RL algorithms like PPO would nonetheless require new trainers that decouple rollout collection from learning and manage reference policies and value heads, but these can reuse ESPnet2's distributed infrastructure.

Overall, viewing "RL alignment and control hooks" as a taxonomy dimension highlights that NeMo and ESPnet2 are optimised for supervised fine-tuning with varying degrees of extensibility, but lack first-class RL alignment hooks, standard reward abstractions, and streaming-compatible RL training loops for STT. Speech-LM alignment frameworks such as Align-SLM and SpeechLM, considered later in the survey, natively support preference-based optimisation on speech signals but target generative speech tasks rather than CTC- or WER-optimised transcription.

C. Summary Table

Table II summarises the taxonomy dimensions across NeMo and ESPnet2 at a high level.

V. NVIDIA NeMo for STT Fine-Tuning

A. Framework Overview

NVIDIA NeMo is a modular toolkit for developing speech, language, and multimodal models, with a mature automatic speech recognition (ASR) stack built around Conformer and FastConformer encoders paired with CTC or RNN-T decoders. ASR models are configured through YAML files that specify encoder and decoder architectures, tokenizers, optimizers, and schedulers, and are trained using PyTorch Lightning trainers that handle distributed data parallelism, mixed-precision training, checkpointing, and validation. The data layer relies on manifest files in JSON format containing audio paths and transcripts, supports tarred datasets for large-scale training, and implements bucketing by utterance length together with augmentation techniques such as SpecAugment, time-stretching, and pitch shifting to improve robustness.

FastConformer extends the Conformer design with aggressive temporal downsampling via depthwise-separable convolutions and optional long-context attention, enabling efficient training on long-form audio, while CTC and RNN-T decoders provide non-autoregressive and streaming-capable recognition modes, respectively. Loss computation is encapsulated inside the ASR model classes, which support standard CTC loss, RNN-T loss with a joint network over encoder and prediction states, and hybrid or intermediate-CTC variants that attach auxiliary CTC losses to intermediate encoder layers. A generic fine-tuning entry-point script orchestrates adaptation by loading pretrained checkpoints from local files or model hubs, handling tokenizer and vocabulary alignment, setting up training and validation manifests, and running a validation loop that periodically computes WER and preserves the best-performing checkpoints.

B. Fine-Tuning Strategies

NeMo supports a spectrum of fine-tuning strategies ranging from full model adaptation to parameter-efficient approaches that update only small subsets of parameters. In the full fine-tuning regime, all weights of the encoder and decoder are updated on the target-domain corpus using optimizers such as AdamW or SGD with Hydra-configured learning-rate schedules (e.g., warm-up followed by cosine decay), providing strong adaptation at the cost of high memory usage and a larger risk of overfitting when domain data are limited. For a typical 120 M-parameter FastConformer model, full fine-tuning involves updating all ~ 120 M parameters, whereas LoRA with rank 32 targeted at encoder linear layers reduces the number of trainable parameters to roughly 1–2 M ($\approx 1.7\%$), and adapter modules with moderate bottleneck sizes yield 3–5 M trainable parameters ($\approx 3\%$).

Parameter-efficient fine-tuning (PEFT) in NeMo 2.0 is implemented through "model transforms" that wrap selected

TABLE II
TAXONOMY OF STT FINE-TUNING FRAMEWORKS FOCUSING ON NEMO AND ESPNET2.

Framework	Model Types	PEFT Support	RL / Alignment Hooks
NeMo	Conformer, FastConformer; CTC and RNN-T; streaming and offline	LoRA, adapters, IA3; layer freezing; multi-stage recipes	Custom loss overrides; Lightning callbacks; no native PPO or replay buffer API
ESPnet2	Conformer, Transformer, Branchformer, RNN-T; streaming and offline	Config-level init/freeze; SpecAug, warm-up schedulers; recipe-based domain adaptation	`RLESPnetModel` and `RLTrainer` for REINFORCE; reward-model scorer and JSONL prediction logs

modules at the start of training, turning standard linear layers into LoRA-augmented layers or inserting Houlsby-style adapter bottlenecks and IA3 re-scaling modules. These transforms are configured by specifying target module names or prefixes (e.g., attention projections, feed-forward layers), and can in principle be applied to ASR encoders, although most documentation and examples currently focus on large language models, making ASR-specific PEFT recipes less mature and requiring custom code to integrate with ASR training scripts. Beyond PEFT, NeMo supports domain and speaker adaptation through layer-wise learning-rate schedules and staged freezing, for example by freezing lower encoder blocks for initial epochs before unfreezing them, and by adjusting warm-up steps and base learning rates to stabilise fine-tuning on smaller or more specialised corpora.

In practical domain adaptation workflows, users typically start from a pretrained Conformer or FastConformer checkpoint, load new manifests for the target domain, optionally freeze lower encoder layers, and fine-tune with a reduced learning rate and batch size while monitoring validation WER and saving the best checkpoint according to task-specific metrics. Speaker or environment-specific adaptation can be achieved by further fine-tuning on speaker-clustered or environment-filtered subsets of data, with PEFT methods providing a way to maintain separate lightweight adapters per domain or speaker group while reusing a shared base model. These strategies make NeMo a flexible platform for supervised adaptation, even though out-of-the-box recipes for aggressive continual-learning or replay-based adaptation are not yet standardised.

C. RL/RLHF Compatibility Analysis

NeMo includes dedicated RLHF tooling for large language models, most notably NeMo-Aligner, which provides infrastructure for supervised fine-tuning, reward modeling from preference data, and PPO/GRPO-style RLHF training on text-based LLMs. This stack leverages NeMo's model-transform and trainer abstractions to apply PEFT (e.g., LoRA) to LLMs, integrate custom reward functions, and orchestrate distributed RLHF runs, but it is architected primarily around token-level language models rather than ASR encoders and decoders. Consequently, while NeMo as a whole is RLHF-aware, its ASR-specific components do not expose ready-made RLHF recipes or APIs analogous to those available for LLM alignment.

From an architectural standpoint, integrating reward-based objectives into NeMo's ASR models is feasible but requires targeted modifications. ASR models centralise loss computation in dedicated routines that currently construct CTC, RNN-T, or hybrid objectives from encoder and decoder outputs; replacing or augmenting these losses with reward-based terms (for example, functions of WER, semantic similarity, or downstream task metrics) would involve extending these routines to accept externally computed rewards and to form blended objectives of the form $\mathcal{L}_{\text{total}} = (1-\lambda)\,\mathcal{L}_{\text{sup}} + \lambda\,\mathcal{L}_{\text{RL}}$. Similarly, policy-gradient updates could be approximated by treating negative rewards as pseudo-losses attached to entire utterances or sequences, provided that NeMo's decoders expose log-probabilities of hypotheses and that the training scripts are adapted to switch between teacher-forced and sampled decoding modes. In principle, NeMo's mechanism for programmatically transforming models could also be used to attach additional heads (for example, value estimators) to ASR encoders for critic networks in actor–critic or PPO-style algorithms, though this would require new trainer logic beyond the existing supervised pipelines.

However, several limitations make online and streaming RL for STT non-trivial in NeMo's current ASR stack. First, the PyTorch Lightning trainers are designed around epoch-based supervised training loops rather than decoupled rollout and update phases; implementing on-policy RL (e.g., PPO) or asynchronous off-policy methods would require refactoring trainers to separate inference-only rollout collection from gradient updates and to manage reference policies and replay buffers. Second, NeMo's ASR APIs are geared toward offline batch training and evaluation, with streaming ASR supported at inference time but not tightly integrated into a training loop that could consume live feedback or streaming rewards. Third, while the RLHF components for LLMs provide abstractions for reward models and rollout engines, they do not currently offer CTC- or RNN-T-aware reward computation, beam-search integration, or per-utterance WER-based rewards for ASR, implying that substantial glue code would be needed to adapt them for speech recognition tasks.

In summary, NeMo offers a strong foundation for supervised STT fine-tuning and a separate, well-engineered RLHF stack for LLMs, but there is a gap between these two worlds: ASR models lack first-class RL alignment hooks, standard reward abstractions, and streaming-compatible training loops, making NeMo a capable but incomplete substrate for building fully

self-adaptive, RL-driven STT systems today.

VI. ESPNET AND ESPNET-SPEECHLM

A. ESPnet Overview

ESPnet and its successor ESPnet2 form a general-purpose end-to-end speech processing toolkit that supports ASR, text-to-speech, speech translation, enhancement, diarization, and other tasks through a unified recipe-based interface. For ASR, ESPnet2 provides a rich catalog of encoder–decoder architectures, including Conformer, Transformer, Branchformer, RNN-T, and streaming Conformer variants, instantiated via YAML configuration files and orchestrated by corpus-specific shell scripts that encode standard multi-stage recipes such as the LibriSpeech pipeline. Hybrid CTC/attention models are the default, with a scalar coefficient blending frame-synchronous CTC loss and sequence-to-sequence attention loss, and optional external language models integrated via shallow fusion during decoding. Recipes are organised into multi-stage pipelines controlled by stage flags, covering data download and preparation, subword (BPE) tokenizer training, optional language-model training, ASR training, checkpoint averaging, and decoding and scoring. The LibriSpeech ASR recipe, for example, trains a Conformer encoder with a Transformer decoder and CTC/attention hybrid loss, then performs N-best checkpoint averaging and beam-search decoding with configurable beam size and loss weights, before computing WER using standard tools from the SCTK suite or equivalent Python implementations. Pretrained models hosted in the official ESPnet model repository can be downloaded and evaluated directly via subset scripts that replicate the decoding stage, enabling rapid experimentation and validation without full retraining.

In the taxonomy developed in this survey, ESPnet2 is treated as one of the two primary STT fine-tuning frameworks, alongside NeMo, while ESPnet-SpeechLM is considered as related speech language modeling and RLHF tooling rather than as a core STT system.

B. Fine-Tuning and Adaptation Mechanisms

ESPnet2's fine-tuning and domain adaptation workflows are built on top of a flexible training core that implements the generic optimisation loop for ASR and related tasks, together with a set of configuration hooks that control initialisation, parameter freezing, schedulers, and robustness modules. The core training loop accumulates gradients over multiple mini-batches before each optimizer step, applies global gradient clipping, and steps one or more optimizers and batch-step schedulers that include an initial warm-up phase, while epoch-level schedulers adjust learning rates based on validation metrics. Model construction is handled by the ASR task's model-building routine, which composes front-ends, SpecAugment, encoders, decoders, and CTC heads from the corresponding sections of the YAML configuration file, providing a clean mapping from configuration files to concrete architectures.

Domain and speaker adaptation rely on a combination of initialisation and freezing controls, scheduler choices, and data selection. Configuration keys allow fine-tuning to start from existing checkpoints and specify which modules (for example, lower encoder layers) should remain frozen, enabling partial adaptation of higher layers to new domains. Fine-tuning templates typically reduce the learning rate, shorten warm-up schedules, decrease the effective batch size, and increase SpecAugment strength relative to full training recipes, reflecting the smaller size and higher overfitting risk of domain-specific datasets. Corpus-specific configurations for LibriSpeech, AISHELL, and other benchmarks demonstrate standard choices—Conformer encoders with hybrid CTC/attention decoders, Adam optimizers, and warm-up learning-rate schedulers—that can be reused or adapted for new domains and languages.

On the evaluation side, ESPnet2 provides an extended validation and logging pipeline that is especially relevant for continual-learning and RL-style experiments. Custom inference utilities can run beam-search decoding over subsets of data and emit JSONL logs containing utterance identifiers, hypotheses, references, per-utterance WER, beam scores, and token sequences. These logs can be used to compute aggregate WER and CER, as well as to simulate reward signals (for example, $1 - \text{WER}$) and per-utterance log-probabilities for analysis or offline policy-gradient experiments, without modifying the core trainer.

C. RLHF and Preference-Based Learning

Beyond supervised ASR, the ESPnet ecosystem now includes ESPnet-SpeechLM, a toolkit for speech language models and voice agents that extends ESPnet2's infrastructure to speech-conditioned generative models and RLHF-style alignment. SpeechLM-style models operate over sequences of speech tokens (e.g., discrete units learned from audio) and support tasks such as speech continuation, dialog generation, and multi-modal interaction, with recipes that incorporate preference-based objectives and reinforcement learning from AI feedback (RLAIF) to align outputs with semantic and stylistic targets. These capabilities position ESPnet-SpeechLM as speech-LM alignment tooling analogous to text-focused RLHF libraries, rather than as an ASR or WER-optimised STT framework.

Within the ASR stack itself, ESPnet2 offers an explicit scaffold for reinforcement learning extensions through dedicated RL-aware model and trainer variants. In this design, the trainer injects a scalar weight into each batch, and the model's main computation routine evaluates both the conventional supervised loss (CTC/attention) and a policy-gradient loss term based on decoded hypotheses and per-utterance rewards, blending them via a template of the form $\mathcal{L}_{\text{total}} = (1 - \lambda)\,\mathcal{L}_{\text{CE}} + \lambda\,\mathcal{L}_{\text{PG}}$. Per-utterance WER can be computed inside the model or via external scorers, and JSONL prediction logs produced by the logging-oriented inference utilities mirror the same reward computations, enabling offline analysis and debugging of reward distributions before committing to full RL training runs.

Despite these advances, several gaps remain when considering RLHF and preference-based learning specifically for production STT. ESPnet-SpeechLM's RLHF capabilities are tailored to speech language models and generative tasks rather than to CTC- or transducer-based ASR optimised for WER, so they do not directly expose WER-aware reward models, CTC-aligned decoding objectives, or streaming constraints. Within ESPnet2, the REINFORCE-style integration is well-suited for initial experiments but does not yet provide complete support for more sample-efficient algorithms such as PPO, which would require dedicated rollout buffers, reference policies, critic networks, and separate rollout and learning phases. Furthermore, while ESPnet2 offers streaming Conformer architectures and recipes, RL and RLHF integration remains primarily offline and batch-oriented, lacking mechanisms for real-time reward ingestion, latency-aware rewards, or deployment-friendly adaptation loops that can operate under strict production constraints.

Overall, ESPnet2 provides one of the most extensible supervised foundations among current STT toolkits, with explicit hooks for reward-based optimisation, while ESPnet-SpeechLM contributes complementary speech-LM RLHF capabilities that inform but do not replace ASR-focused fine-tuning. Turning these ingredients into a turnkey solution for self-adaptive STT still requires bridging gaps around WER- and CER-centric reward modeling, streaming and low-latency operation, and robust deployment workflows that can safely run RL-driven updates in production environments.

VII. Survey of RL/RLHF Frameworks Relevant to STT

A. Taxonomy of RL/RLHF Frameworks

From the perspective of speech-to-text self-adaptation, existing reinforcement learning frameworks can be broadly grouped into three categories: (i) language-model-focused RLHF toolkits designed for aligning large language models with human or AI preferences, (ii) speech-language-model alignment stacks that extend RLHF ideas to speech-conditioned generative models, and (iii) general-purpose RL libraries that target a wide range of control and decision-making problems. LM-focused frameworks, such as VERL, OpenRLHF, DeepSpeed-Chat, Hugging Face TRL, TRLX, and RL4LMs, provide end-to-end pipelines for supervised fine-tuning, reward modeling, and on-policy or preference-based optimisation of Transformer policies operating over text token sequences. Speech-LM alignment stacks, exemplified by Align-SLM and SpeechLM, operate over discretised speech tokens and adapt RLHF-style pipelines (e.g., DPO, RLAIF) to speech-to-speech or speech-conditioned generation. General RL libraries, exemplified by Ray RLlib and related toolkits, offer algorithm-agnostic infrastructure (environments, policies, replay buffers, distributed rollouts) that can, in principle, host speech recognition environments but do not natively encode ASR- or CTC-specific structure.

Within the LM-focused group, frameworks differ in the granularity of abstraction they provide. Systems like VERL and OpenRLHF emphasise scalable, production-grade RLHF and RLVR (reinforcement learning from verifiable rewards), with rich support for parallel rollout engines, multi-GPU training backends, and an extensive menu of RL algorithms (PPO, GRPO, REINFORCE variants, DPO and related objectives). In contrast, higher-level libraries such as TRL, TRLX, and RL4LMs wrap Hugging Face Transformers models with simplified APIs for SFT, reward modeling, and a smaller set of RLHF or preference-optimisation methods, trading off some systems flexibility for ease of use. Speech-LM stacks such as Align-SLM and SpeechLM inherit many of these RLHF abstractions, but apply them to speech tokens rather than text, providing concrete examples of how preference-based training and AI feedback can be extended into the audio domain even though they do not target WER-optimised transcription. General RL libraries like RLlib expose a wide collection of on-policy, off-policy, model-based, and multi-agent algorithms, together with environment and policy abstractions that are agnostic to the underlying domain, making them candidates for custom STT environments but with minimal speech-specific tooling out of the box.

B. High-Level Framework Summaries

VERL, AReaL, MILES, SMILE. VERL (Volcano Engine Reinforcement Learning) is a modular RLHF and RLVR framework that supports a wide spectrum of post-training algorithms, including PPO, GRPO, GSPO, ReMax, REINFORCE++, leave-one-out variants, and replay-augmented methods such as PF-PPO, along with standard supervised fine-tuning for the initial stage. It distinguishes between RLHF, where reward models trained from preference data provide scalar feedback, and RLVR, where verifiable rewards such as unit-test pass rates or symbolic checkers are used, and offers extensions (verl-tool, verl-agent) for multi-turn, tool-using agents interacting with external systems like code runners or search APIs. AReaL, MILES, and SMILE, while differing in focus, similarly explore scalable RL or RLHF for large models with an emphasis on high-throughput rollouts, asynchronous or mixed online–offline training regimes, and improved sample efficiency, positioning them as research-oriented frameworks rather than turnkey STT solutions.

OpenRLHF, TRL, DeepSpeed-Chat, TRLX, RL4LMs. OpenRLHF is a production-ready RLHF/RLVR framework that tightly couples Ray for distributed orchestration, vLLM for efficient rollout generation, and DeepSpeed for memory-optimised training, enabling alignment runs for models exceeding tens of billions of parameters. Its architecture decomposes into a Ray-based controller, vLLM rollout engines, DeepSpeed training backends, an agent execution layer abstracting token-in/token-out interactions, and CLI workflows that cover SFT, reward modeling, PPO-based RLHF, and offline methods such as DPO and KTO, with support for custom rewards and external environments. DeepSpeed-Chat provides a closely related stack focused on DeepSpeed-integrated RLHF training, while Hugging Face TRL, TRLX, and RL4LMs expose higher-level Python APIs for SFT, reward modeling,

PPO- or GRPO-style RLHF, and direct preference optimisation on top of Hugging Face Transformers checkpoints, prioritising developer ergonomics over full-stack systems control.

Speech-LM alignment: Align-SLM and SpeechLM. Align-SLM is a textless spoken language modeling framework that trains discrete-unit speech language models using preference-based alignment, typically via DPO or related objectives applied to AI-generated or human-generated preference pairs over speech continuations. Its pipeline mirrors RLHF for text LMs—supervised pretraining, preference data construction, and preference-optimisation or RL-style updates—but operates on speech tokens and targets generative tasks such as speech continuation, style control, or dialog-like interactions rather than transcription. Related SpeechLM-style toolkits adopt similar speech-tokenisation and alignment strategies, sometimes incorporating reinforcement learning from AI feedback (RLAIF) over semantic or prosodic quality metrics. For this survey, Align-SLM and SpeechLM are therefore treated as speech-LM alignment case studies: they demonstrate how preference-based objectives and AI feedback can be applied directly to speech, offering design signals for reward definitions, tokenisation schemes, and training loops that could, in principle, inform STT-focused alignment even though they are not STT frameworks and do not optimise WER or CTC objectives out of the box.

Ray RLlib and generic RL libraries. Ray RLlib is a general-purpose RL library built on the Ray distributed execution framework, offering more than twenty algorithms across on-policy (PPO, A2C/A3C, IMPALA), off-policy (DQN, Rainbow, SAC, TD3), model-based (DreamerV3, MuZero), multi-agent, and offline RL paradigms. Its architecture includes modular algorithm implementations, flexible environment and policy interfaces (Gymnasium, PettingZoo, custom vectorised environments), a distributed execution layer that scales rollouts and training across clusters, replay utilities for offline and off-policy learning, and callback APIs for curriculum learning and dynamic reward shaping. Although RLlib has been successfully applied to high-throughput environments such as Atari, MuJoCo, and multi-agent StarCraft, and supports GPU acceleration, mixed precision, and gradient compression, it does not provide built-in abstractions for ASR-specific constructs such as CTC decoding, beam search, or WER-based rewards; these must be implemented as custom environments and reward functions by the user.

C. STT Integration Gap (Teaser)

When viewed through the lens of speech-to-text self-adaptation, the surveyed RL and RLHF frameworks reveal a common integration gap. LM-focused RLHF toolkits assume discrete text-token action spaces and autoregressive decoding, with reward models trained on natural language preferences, but lack native support for audio inputs, CTC or RNN-T decoding, or speech-specific error metrics such as WER and CER as first-class reward signals. Speech-LM alignment stacks such as Align-SLM and SpeechLM demonstrate that preference-based training can be extended to speech tokens, but they target generative continuation and conversational tasks rather than WER-optimised transcription, and similarly do not expose ASR-specific objectives or streaming constraints. General RL libraries like RLlib offer the flexibility to define custom environments, but they provide no pre-packaged audio or ASR environments, no specialised replay-buffer schemes for long, variable-length utterances, and no ready-made connectors to established STT frameworks such as NeMo or ESPnet.

As a result, applying these frameworks to STT today requires substantial glue code: wrapping NeMo or ESPnet models as RL policies, exposing log-probabilities and hypotheses to the RL loop, designing reward functions over WER or downstream task performance, and managing rollouts and replay in a way that respects the computational and latency constraints of speech workloads. These integration burdens, together with the absence of streaming-aware, CTC-conscious abstractions, highlight the need for STT-specific RL alignment layers that can bridge between mature ASR toolchains and the powerful but text-centric or speech-generative RL/RLHF infrastructure surveyed above.

VIII. Cross-Framework Mapping: STT vs. RL/RLHF Components

A. Conceptual Mapping

To reason about how STT toolchains and RL/RLHF frameworks might interoperate, it is useful to view them through a shared set of abstractions. On the STT side, frameworks such as NeMo and ESPnet2 expose components including data loaders and manifests, acoustic and sequence models, training loops, and evaluation metrics. On the RL/RLHF side, frameworks such as VERL, OpenRLHF, TRL, TRLX, RL4LMs, and RLlib decompose the problem into environments, policies, reward functions, replay buffers, and optimizers or RL trainers.

Conceptually, the STT data loader and manifest system correspond to the environment's observation generator: they select audio segments and associated metadata (references, domains) that define initial states and contextual information for each trajectory. The ASR model (e.g., Conformer encoder plus CTC or attention decoder) plays the role of the policy, mapping acoustic features to token sequences, while beam search or CTC decoding correspond to the policy's action-selection mechanism over sequences of discrete actions. Evaluation metrics such as per-utterance WER, CER, or semantic scores can be interpreted as reward functions over completed trajectories, possibly composed with latency or cost terms in production settings, and JSONL prediction logs in ESPnet2 already instantiate this mapping by recording hypotheses, references, log-probabilities, and WER for each utterance.

From the RL/RLHF standpoint, STT training loops and schedulers approximate the role of the optimizer or RL trainer. NeMo's PyTorch Lightning trainers and ESPnet2's task-specific training classes drive gradient updates, blending supervised losses (CTC/attention) with optional policy-gradient terms when RL is enabled. Replay buffers and off-policy storage in RL frameworks correspond to potential extensions of STT data pipelines, where curated subsets of past

audio–transcript pairs or logged trajectories would be mixed with new data to stabilise continual learning; some of this functionality can be emulated via custom data loaders in NeMo and the reward-logging pipeline in ESPnet2, but it is not yet a first-class abstraction. Finally, RL-specific components such as reference policies and value networks map naturally onto frozen STT checkpoints for KL regularisation and auxiliary heads on encoders for critic estimates, but these are not currently standard in mainstream ASR recipes.

B. Integration Pain Points

Despite the apparent conceptual alignment, several structural mismatches complicate direct integration. One major issue is training-loop ownership: RLHF frameworks expect to orchestrate the full rollout–update cycle, while STT toolkits encapsulate training in monolithic scripts or trainers that assume purely supervised objectives. In NeMo, for example, loss computation and optimiser steps are baked into PyTorch Lightning modules and configuration-driven scripts, so on-policy RL or asynchronous rollouts would require refactoring these trainers to decouple inference-only phases from gradient updates. ESPnet2 partly addresses this by embedding RL logic inside the existing trainer, but full PPO-style separation of rollout and learning still demands additional scaffolding. By contrast, RL libraries such as VERL, OpenRLHF, and RLlib assume central control over environment stepping and policy updates, making it non-trivial to plug in an STT toolkit that wants to manage its own optimisation schedule and checkpointing.

A second pain point lies in observation and action mismatches. RLHF toolkits are predominantly text-centric: policies operate over prompt–token sequences, and environments supply text observations that fit neatly into LLM context windows, whereas STT pipelines consume raw waveforms or spectrograms and produce transcripts via CTC or encoder–decoder mechanisms. Bridging this gap requires wrapping ASR models as policies that expose log-probabilities over token sequences, treating complete utterance transcriptions as episodic trajectories, and defining rewards over WER, CER, or semantic quality—none of which are first-class citizen abstractions in existing RL frameworks. Moreover, streaming STT introduces partial observations and low-latency constraints that standard RLHF rollouts, which assume full prompts and responses, are not designed to handle.

Finally, performance and deployment constraints differ markedly between STT and text LMs. RLHF systems for LLMs optimise throughput in tokens per second using engines like vLLM or SGLang, and can often tolerate higher latencies during training because rollouts operate on short text prompts. STT workloads, by contrast, process long and variable-length audio segments, often under real-time or near-real-time constraints, with heavy encoder computation dominating cost and latency; naive application of RLHF frameworks to speech would likely yield prohibitively slow rollouts and infeasible GPU budgets without speech-specific optimisations. Additionally, production STT services have strict availability and regression constraints, so integrating RL-driven updates into deployment pipelines requires careful orchestration of evaluation, rollback, and cost-aware scheduling that current RLHF stacks, designed primarily for offline training jobs, do not natively provide.

C. Illustrative Mapping Table

Table III provides a high-level, illustrative mapping between core STT and RL/RLHF components, highlighting where NeMo, ESPnet2, and generic RL frameworks already align and where additional glue or new abstractions are required.

IX. Experimental Comparison of STT Fine-Tuning Frameworks

TABLE III
CONCEPTUAL MAPPING BETWEEN STT AND RL/RLHF COMPONENTS, AND CURRENT SUPPORT IN NEMO, ESPNET2, AND GENERIC RL FRAMEWORKS.

Concept	STT Realisation	RL/RLHF Role	Current Support
Data / Environment	Manifests, data loaders	Environment, sampler	NeMo/ESPnet native; RLlib env APIs
Policy model	ASR encoder–decoder	RL policy π_θ	Both; RL side text-centric
Rewards / metrics	WER, CER, semantic scores	Reward function $R(x, y)$	ESPnet2 logs; RLHF reward models
Trainer / optimiser	CE/CTC trainers, schedulers	RL trainer, PPO/DPO, etc.	Both; limited cross-wiring
Replay / buffer	Custom loaders, logs (optional)	Replay buffer, trajectory store	RL side only; STT via custom code
Reference / critic	Frozen checkpoints, extra heads	Ref. policy, value network	Ad hoc in STT; native in RLHF

X. GAP ANALYSIS AND RESEARCH OPPORTUNITIES

A. Gaps in STT Fine-Tuning Frameworks

Although NeMo and ESPnet2 provide strong support for supervised ASR training and domain adaptation, they lack native interfaces for online or on-policy adaptation driven by runtime feedback. NeMo's ASR trainers are built around epoch-based PyTorch Lightning loops that assume static datasets and fixed objectives; integrating on-policy RL requires overriding internal loss functions and restructuring training scripts to decouple inference-only rollouts from gradient updates, which is not supported out of the box. ESPnet2's `RLESPnetModel` and `RLTrainer` demonstrate that REINFORCE-style updates can be embedded into the existing trainer, but more advanced algorithms like PPO still demand new trainers, value heads, and rollout buffers, indicating that online adaptation remains a niche extension rather than a first-class capability.

Support for continual learning and replay buffers is similarly limited. NeMo's manifest-based data loaders and dynamic bucketing are well-suited to static or periodically refreshed datasets, but experience replay requires custom data loaders, reward tracking per sample, and prioritised sampling—features that must be engineered separately by users. ESPnet2 recipes encode strong single-domain and multi-corpus training practices, but do not explicitly expose domain-aware replay protocols, backward-transfer metrics, or drift-aware sampling strategies needed for robust sequential adaptation under domain shift. As a result, researchers must construct ad hoc replay buffers and drift protocols on top of these frameworks, rather than relying on built-in continual-learning abstractions.

Finally, existing STT toolkits do not offer standardised reward interfaces over speech-specific metrics. While both NeMo and ESPnet2 compute WER and CER in evaluation pipelines, these metrics are treated as diagnostic outputs rather than differentiable or pluggable reward signals that can be used in training objectives. ESPnet2's RL scaffold introduces an `rl_weight` parameter and a blended loss template for combining cross-entropy and policy-gradient terms, but reward functions (e.g., $1 - \text{WER}$, semantic similarity) are still ad hoc Python code rather than part of a unified reward API spanning training and decoding. This lack of standard reward abstractions over WER, CER, diarization error rate, and domain-specific metrics makes it difficult to share or reproduce RL-style STT experiments, and hinders integration with external RL/RLHF engines.

B. Gaps in RL/RLHF Frameworks for STT

Conversely, RL and RLHF frameworks have been optimised primarily for text-based large language models and make assumptions that do not transfer cleanly to speech recognition. Full-stack systems such as VERL, OpenRLHF, DeepSpeed-Chat, AReaL, MILES, and SMILE, and research libraries like TRL, TRLX, and RL4LMs all assume discrete text-token action spaces, autoregressive decoding, and reward models trained on textual preferences, with no native support for audio inputs, CTC or RNN-T decoding, or WER/CER as first-class reward signals. Generic RL libraries such as Ray RLlib provide domain-agnostic environment and policy abstractions, but do not include built-in ASR environments, speech features, or streaming constraints, leaving all speech-specific logic to end users.

There is also no turnkey integration between these RL/RLHF frameworks and mainstream STT toolkits like NeMo and ESPnet2. Although RL libraries allow custom environments, wrapping an ASR model as a policy requires implementing audio feature extraction, decoding, reward computation, and logging, as well as handling long variable-length trajectories and high inference cost. Similarly, while NeMo and ESPnet2 can, in principle, expose hooks for external RL controllers, there are no standard adapters, APIs, or examples demonstrating end-to-end pipelines where RLHF frameworks drive NeMo/ESPnet2 models in a closed loop. This integration gap significantly raises the engineering barrier to deploying RL-driven STT adaptation compared to text-based RLHF, where policy, reward, and rollout components are already tightly integrated.

C. Design Requirements for Self-Adaptive STT

The preceding gaps motivate a set of design requirements for a unified, self-learning STT framework that bridges ASR toolchains and RL/RLHF infrastructure. First, such a framework should provide speech-native environment abstractions that treat STT models as policies acting on audio streams, with well-defined observation (features, context), action (token sequences, partial hypotheses), and termination semantics for both offline and streaming scenarios. These abstractions should integrate cleanly with existing ASR toolkits, reusing their data loaders, models, and decoders while exposing them through RL-compatible interfaces rather than re-implementing the entire stack.

Second, the framework should offer standardised reward interfaces over STT metrics and semantic measures. Reward functions should be able to combine WER, CER, diarization and verb error rates, domain-specific error rates, latency and cost penalties, and potentially downstream task performance into scalar signals usable by RL algorithms, with consistent APIs across training, validation, and decoding pipelines. Support for both learned reward models (e.g., preference-based quality predictors) and verifiable rewards (e.g., ground-truth WER on curated subsets, or consistency checks) would enable flexible alignment objectives analogous to RLHF and RLVR in text domains.

Third, a self-adaptive STT framework should incorporate continual-learning and replay mechanisms as first-class components. This includes replay buffers for audio–transcript trajectories, drift-aware sampling strategies, backward-transfer metrics, and scheduling policies that decide when to trigger adaptation based on performance degradation and cost constraints. Rather than relying solely on periodic manual retraining, the system should support both batch and incremental updates, with pluggable controllers that can implement simple heuristics or full RL-based scheduling policies.

Finally, the overall design should be modular and interoperable rather than monolithic. ASR models, reward sources, replay buffers, schedulers, and RL optimisers should be composable, allowing practitioners to reuse existing NeMo or ESPnet2 recipes, plug into RL/RLHF engines where appropriate, and deploy in a range of operational modes (offline experiments, shadow training, or live adaptation) without committing to a single algorithm or vendor. By keeping these design requirements abstract—focusing on interfaces, capabilities, and integration patterns rather than specific architectures—this survey sets the stage for future work to instantiate concrete self-adaptive STT systems that satisfy them.

XI. Open Challenges and Future Directions

XII. Conclusion

Acknowledgment

References